fulfill the request:

- Create and return a new instance of the service.
- Return an existing instance of the service.

The *lifetime* of a service controls the behavior of the DI container with respect to these two options. You define the lifetime of a service during DI service registration. The lifetime dictates when a DI container reuses an existing instance of the service to fulfill service dependencies and when it creates a new one.

DEFINITION The *lifetime* of a service is how long an instance of a service should live in a container before the container creates a new instance.

It's important to get your head around the implications for the different lifetimes used in ASP.NET Core, so this section looks at each lifetime option and when you should use it. In particular, you'll see how the lifetime affects how often the DI container creates new objects. In section 9.4.4 I'll show you an antipattern of lifetimes to watch out for, in which a short-lifetime dependency is captured by a long-lifetime dependency. This antipattern can cause some hard-to-debug issues, so it's important to bear in mind when configuring your app.

In ASP.NET Core, you can specify one of three lifetimes when registering a service with the built-in container:

- *Transient*—Every time a service is requested, a new instance is created. Potentially, you can have different instances of the same class within the same dependency graph.
- *Scoped*—Within a scope, all requests for a service give you the same object. For different scopes, you get different objects. In ASP.NET Core, each web request gets its own scope.